Final project

Workshop: Software Product

Project name: Boolean Logic

Project goal: The goal of the project is a playful and interactive application for understanding and improving mental logic and Boolean algebra.

Introduction

Boolean Logic is a web application that includes a strategic board game, two challenge games that involve analyzing Boolean formulas and a graphical simulator of building logic circuits using logic gates. The goal of the project is to improve logical thinking in general and Boolean algebra in particular in a fun and challenging game-like way and through experimentation in building logic circuits.

This project is suitable for the Software Product workshop because its product will be a web application with a server side and a client side and several features (games and logic circuit simulator).

Project description

As stated, the app will include four main features: a strategic board game, two challenge games that involve analyzing Boolean formulas and a logic circuit simulator with optional challenge mode.

Feature Description:

The board game:

The board game is played on a 7x7 grid board, so you can think of it as a chessboard where all the black squares are logic gates squares and all the white squares are bits square.

The game has two different game modes- for two players (or against the computer) and for one player.

mode 1 – two players:

At the beginning of the game, all the squares of the logic gates are filled with randomly selected logic gates. Each player in turn chooses a bit square and chooses a bit that he places in the square (0 or 1). This bit is colored in the player's color. Each logic gate receives two values: a value for the two bits above and below it, and a value for the two bits to its right and left. When a logic gate receives a value, the bits for which this value is valid (the two bits on either side of the gate or the two bits above and below the gate) earn or lose points to the player who placed them (the color of each bit is the same as the color of the player who placed it) according to the following rules: If the bit is equal to the gate value - the

player earns a point, if the bit is different from the gate value - the player loses a point. The game ends when all the squares are filled and the winner is the player with the highest number of counters, in this mode you can play against the computer or one-on-one online.

A sketch of some game stages in this mode:

start of game:

	or		and		or	
nand		or		xor		and
	and		nor		nxor	
xor		nand		nor		or
	or		and		nor	
nor		xor		nxor		and
	or		nand		nor	

red score: 0 blue score: 0

middle of game:

	or	1	and	1	or	
nand		or		xor		and
	and	0	nor		nxor	
xor		nand		nor		or
	or		and		nor	
nor		xor		nxor		and
	or		nand		nor	

red score: 2 blue score: 0

As you can see in the example, after three turns in the game, the value of the "and" gate in the top row for the two bits on its sides is 1 and since their value is also one - each of these bits earns a point for the player who placed them, in this case one point for the blue player and one point for the red player, for the "or" gate in the second row - its value for the two bits above and below it is 1, Therefore, the bit above it earns a point for the red player because its value is also 1 and is equal to the gate value, the bit below the "or" gate deducts a point for the blue player because it is 0 and is different from the gate value (which is 1), meaning that the blue player has so far earned a point and lost a point, so he currently has 0 points, and the red player has earned two points.

End of game:

1	or	1	and	1	or	0
nand	0	or	1	xor	1	and
0	and	0	nor	0	nxor	0
xor	0	nand	0	nor	1	or
1	or	1	and	1	nor	0
nor	1	xor	1	nxor	0	and
0	or	1	nand	0	nor	0

red score: 10 blue score: -7

Mode 2 – solo (puzzle mode):

In this mode, the game is the same as described, only there is one player who needs to fill the entire board and reach the maximum score that can be achieved on the current board (i.e. with the current logical gates that have been drawn). Once the player has filled the board, the computer tells him by how many points he is far from an optimal solution (i.e. a solution of filling bits in the board that will achieve the maximum number of points that can be achieved on this board), and the player can keep changing the bits on the board until he reaches an optimal solution.

The logic circuit simulator:

Mode 1 – free simulator:

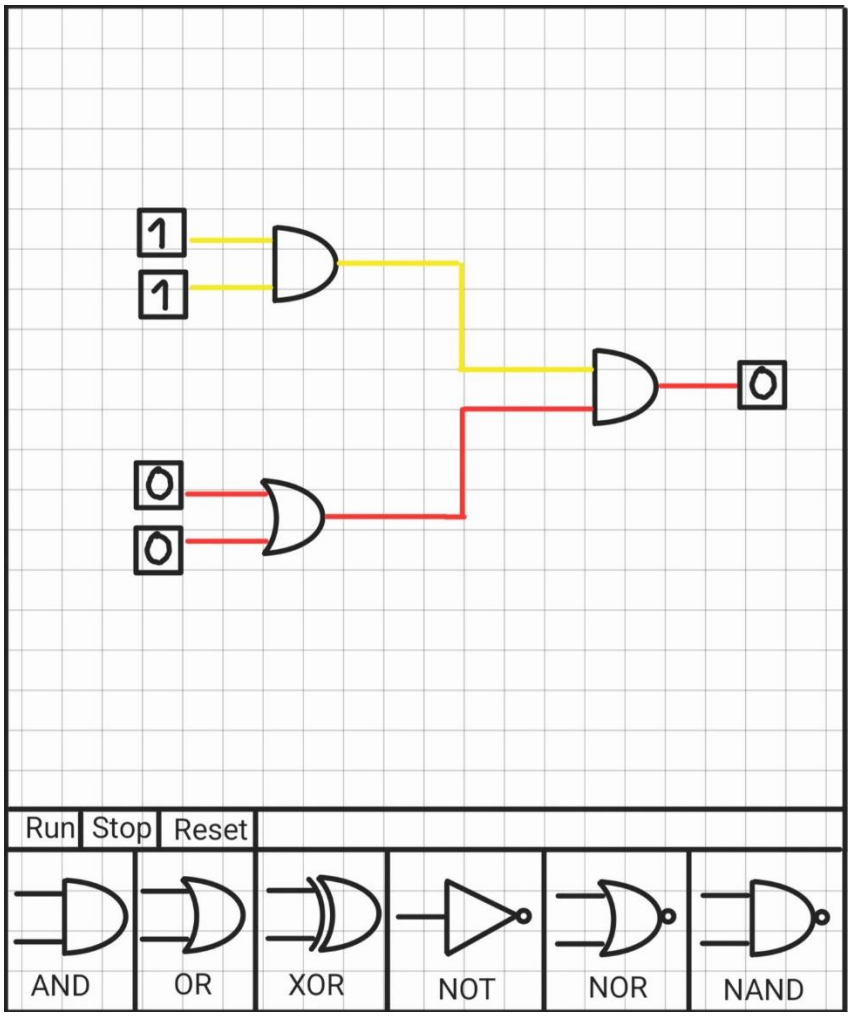
The logic circuit simulator provides a free-form workspace in which users can build logic circuits. Users can drag logic gates from a ToolBar containing all the logic gates, wires to connect the gates, and wire split points onto a canvas, connect them using wires, and assign binary values to input nodes. The system visualizes signal propagation in real time, showing where signals flow, stop, or branch.

Mode 2 – challenge mode:

The challenge mode features three difficulty levels (easy, medium, and hard). The user is presented with a truth table—involving 3, 4, or 5 variables, depending on

the level—and must design a logic circuit that implements that table using the minimum number of gates. After building the circuit, the user submits the solution and receives feedback indicating whether the circuit correctly implements the function and whether the software found a solution requiring fewer gates than the user's design. The user can also click a button to view the solution with the minimum gate count found by the software.

Partial sketch of the simulator:



In the sketch, a red wire represents a wire that has 0 passing through it and a yellow wire represents a wire that has 1 passing through it.

Boolean Formula Evaluation Game:

This game is for one or two players.

In the game, the player or both players will be given a Boolean formula with Boolean variables and a placement, and will have to solve the formula.

The score in this game will be based on the correctness of the solution and the time it took to reach the solution

Example of a formula as given in this game:

$$(\bar{x} \wedge \bar{y} \wedge \bar{z}) \vee (\bar{x} \wedge \bar{y} \wedge z) \vee \overline{(x \wedge z)} =$$

with the placement: x = 1, y = 0, z = 0

Boolean Formula Equivalence Game:

This game is for one or two players.

In the game, the player or both players will be given two Boolean formulas with Boolean variables like the example formula we showed in the previous section, the formulas look syntactically different but may or may not be logically equivalent. The player's task is to determine whether the two formulas are Logically equivalent or not equivalent.

The score in this game will be based on the correctness of the solution and the time it took to reach the solution

App visibility:

The app's main screen is a menu featuring three buttons: one for the formula games, one for the board game, and one for the simulator. Selecting an option leads to sub-menus corresponding to the specific game modes for that feature. for instance, clicking the board game button opens a sub-menu where the user selects the game mode (solo, online one-on-one or one-on-one against the computer). If the "against the computer" mode is chosen, the user proceeds to a sub-menu to select the difficulty level (easy, medium, or hard). The same logic applies to the other two features, based on their respective game modes as previously described in the document.

Project design

The app follows a client–server architecture, the project is organized as a monorepo managed with npm workspaces, consisting of three workspaces: client, server, and shared. The shared workspace contains all modules that are used by both the client and the server, and is imported by both as a local package.

The system is divided into three top-level architectural layers:

Client (Frontend) – responsible for user interaction, visualization, local logic execution, and offline gameplay.

Server (Backend) – responsible for online multiplayer coordination, authoritative validation, and synchronization between remote clients.

Shared – responsible for housing all logic that is common to both the client and the server, packaged as an independent workspace within the monorepo.

Main modules:

Logic Gate Handler:

Purpose: The Logic Gate Module provides the fundamental, deterministic computation of Boolean logic gates. It represents the lowest-level logical building block of the system and is reused across multiple modules. This module is completely independent of any specific game or user interface.

Components:

Gate Evaluator - Computes the output of a logic gate (AND, OR, XOR, NOT, NAND, NOR, etc.) given one or more binary inputs.
This component encapsulates the truth tables of all supported gates.

Defines a uniform interface for passing inputs into a gate and retrieving outputs.

Module technology – TypeScript, Vitest.

Board Game Engine:

Purpose: The Board Game Engine implements all rules and mechanics of the board-based Boolean logic game.

Components:

Move Validator - Checks whether a player move is legal according to the board rules, cell type restrictions, and turn order.

Score Calculator - Evaluates logic gates affected by a turn and updates player scores based on correctness of outcomes.
Uses the Logic Gate Module for gate evaluation.

Game State Manager - Maintains the full board state, including gate placements, bit values, current turn, and game completion detection, supports changing bit values already set on the board in solo mode only.

Board Initializer - Randomly selects and places logic gates on all gate squares at the start of a new game, drawing from the full set of supported gate types (AND, OR, XOR, NAND, NOR, XNOR, NXOR). Ensures the distribution is uniformly random and that the board is fully initialized before the first player turn.

Optimal Board Solver - Computes the maximum achievable score for a given board configuration, that is, for a fixed set of gate placements. Used exclusively

in Mode 2 to inform the player by how many points their current solution falls short of the optimum.

Module technology – TypeScript, Vitest.

Computer Player Engine:

Purpose: Provides deterministic decision-making for the player-versus-computer mode of the board game. Uses deterministic algorithms such as Minimax, Alpha-Beta pruning, and heuristics.

Components:

Move Generator - Generates all legal moves available for a given board state.

Search Best Move - Explores possible future game states using Minimax with optional Alpha–Beta pruning, and return the move for this turn(relies on the Board Game Engine module to simulate moves and compute the resulting board

state and score after each hypothetical placement).

Difficulty Controller - Parameterizes the behavior of the Search Best Move component according to the difficulty level selected by the player (Easy, Medium,

or Hard). Controls Minimax search depth and adjusts heuristic weights to produce proportionally stronger or weaker play at each level.

Heuristic Evaluator - Assigns scores to non-terminal board states to guide the search algorithm.

Module technology – TypeScript, Vitest.

Board Game UI:

Purpose: A client-side module responsible for rendering the game board and handling all player interaction during a board game session.

Components:

Game Session - The top-level component rendered when a board game session begins. Orchestrates the game loop: alternates turns, applies moves through the Board Game Engine, and displays end-game results. In online mode, sends moves to the server and reflects the authoritative state received via WebSocket. In solo mode it monitors board completion, triggers the Optimal Board Solver when all bit squares are filled, and passes the result to the Solo Feedback Panel. It also handles the iterative loop of bit modification and re-evaluation.

Game Board - the central grid of the screen, composed of two alternating cell types arranged in a fixed pattern. Displays the full board state at all times and updates reactively after each move.

Gate Cell - a fixed, non-interactive cell displaying a logic gate symbol. Its content is determined at game start by the Board Game Engine and does not change during play.

Bit Cell - an interactive cell representing a position where a player may place a bit. When unclaimed it appears blank; when claimed it displays the chosen bit value (0 or 1) filled in the color of the player who claimed it (blue or red). In solo mode a bit cell that already contains a placed bit remains clickable, re-opening the Bit Selector to allow the player to replace the current value with the other one.

Bit selector - Bit Selector — a small popup that appears over a clicked Player Cell, presenting the player with exactly two options: 0 or 1. Upon selection the popup closes and the cell is claimed with the chosen value in the current player's color.

Score Display - a bar rendered below the Game Board showing the current score of the blue player on the left and the red player on the right, updated after each move.

Turn Indicator - Displays which player's turn it currently is. In one-on-one mode it alternates between the two players' names or colors; in solo-vs-computer mode it indicates whether the player or the computer is about to move.

Game Result Display - An end-game screen shown when the board is complete and no further play is possible. Presents the final scores of both sides and declares the winner, or in solo mode confirms that the optimal score has been reached.

Solo Feedback Panel - Shown exclusively in solo mode once the board is fully filled. Displays the player's current total score, the optimal score for the given gate configuration (computed by the Optimal Board Solver), and the gap between them. Updates in real time each time the player modifies a bit.

Rules Dialog - A button permanently visible during play that opens a small modal window containing the game instructions.

Gate Tooltip - Appears on hovering a Gate Cell after a short delay (~500 ms). Shows the gate's current horizontal and vertical computed values, or – for any direction whose bit pair is not yet fully placed.

Module technology – React with TypeScript , CSS Grid for the board layout, CSS custom properties for player color theming.

Logic Circuit Simulator Engine:

Purpose: Allows users to construct and simulate logical circuits visually.

Users can place gates, connect them with wires, assign inputs, and observe real-time signal propagation.

Components:

Circuit State Manager - Maintains the complete data model of the circuit: the set of placed nodes (gates, input sources, output probes), the wires connecting them, and the current signal value on each wire. It is the single source of truth for the circuit's structure and state, and is updated whenever the user adds, removes, or reconnects any element.

Signal Propagator - Given the current circuit structure and the values of all input sources, computes the output signal value of every gate and wire in the circuit. Propagates values from input sources downstream through connected gates to output probes, updating the circuit state with the computed results.

Module technology – TypeScript

<u>Logic Circuit Simulator UI:</u>

Purpose: A client-side module providing an interactive visual workspace where the user constructs Boolean logic circuits by placing gates and

connecting them with wires. It delegates all circuit evaluation to the Logic Circuit Simulator engine and reflects the computed signal values back onto the canvas in real time.

Components:

Circuit Canvas - the main workspace occupying the center of the screen, rendered in SVG. The user drags gates from the Gate Palette onto the canvas, repositions them freely, and connects their pins with wires. Supports panning and zooming.

Circuit Simulator - the top-level component that assembles the simulator screen, owns the circuit state, and coordinates interactions between the canvas, gate palette, and playback controls.

Gate Pallete - a sidebar panel listing all available gate types. Each entry displays the gate name and symbol and can be dragged onto the Circuit Canvas to place a new instance.

Gate Node- the visual representation of a single gate on the canvas. Displays the gate's symbol with input ports on the left side and an output port on the right side. Ports are clickable to begin or complete a wire connection.

Wire - a drawn line connecting an output port of one node to an input port of another. Its color reflects the

live signal value passing through it, updating whenever any input in the circuit changes.

Circuit Input - a special placeable node representing a user-controlled circuit input. Displays the current value (0 or 1) and toggles between them on click. Its output port can be connected to any gate input port.

Circuit Output - a special placeable node that displays the signal value at the point in the circuit where it is connected. Shows 0 or 1, updated in real time.

Wire Split Point - A placeable node on the canvas that allows a single wire to branch into multiple outgoing connections, distributing the same signal to more than one downstream input port.

Simulator Controls - A toolbar containing the Run, Stop, and Reset buttons. Run starts the signal propagation simulation, Stop pauses it, and Reset clears all computed signal values from the canvas and returns the circuit to an uncomputed state.

Module technology – React with TypeScript, SVG (for rendering gate symbols, wires, and ports on the, canvas), @dnd-kit/core (for dragging gates from the Gate Palette onto the canvas), d3-zoom (for panning and zooming the canvas).

<u>Simulator Challenge Engine:</u>

Purpose: Implements all logic for generating circuit challenges, minimizing Boolean expressions to produce reference solutions, verifying user-built circuits against truth tables, and converting minimized expressions into circuit representations compatible with the Logic Circuit Simulator Engine.

Components:

Challenge Generator - Generates a truth table for a given difficulty level (Easy - 3 variables, Medium - 4 variables, Hard - 5 variables). Ensures generated truth tables are non-trivial (not constant, not reducible to a single literal). Runs Circuit Minimizer at generation time and stores the resulting gate count as the reference minimum for that challenge.

Circuit Minimizer - Derives a minimized circuit from a truth table. Runs Quine-McCluskey on the onset (minterms) to produce a minimal SOP, and again on the offset (maxterms) to produce a minimal POS; selects whichever yields fewer gates. Applies post-minimization passes in sequence: complement output check (implements the complement plus a NOT gate if cheaper), XOR/XNOR pattern recognition (replacing two-gate sub-expressions with a single XOR or XNOR gate), and common factor extraction (applying distributivity to share AND sub-expressions across multiple product terms). Returns the expression with the lowest gate count across all passes.

Circuit Verifier - Accepts a circuit state from the Logic Circuit Simulator Engine and a truth table. Enumerates all 2^n input combinations, evaluates the circuit via the Signal Propagator for each, and checks the output against the expected value. Counts all logic gates in the circuit, excluding wires, wire split points, and I/O nodes. Returns a result containing correctness status, gate count, and the list of any failing input combinations.

Solution Builder - Converts the minimized Boolean expression produced by Circuit Minimizer into a circuit-state representation (nodes and wiring) compatible with the Logic Circuit Simulator Engine. Arranges nodes in a structured left-to-right layout for clear display on the Circuit Canvas.

Technology - TypeScript, Vitest.

Simulator Challenge UI:

Purpose: Client-side module for the Simulator Challenge mode. Reuses the Circuit Canvas and Gate Palette from the Logic Circuit Simulator UI and augments them with challenge-specific controls, panels, and feedback components.

Components:

Challenge Session - Top-level component for a challenge session. Loads the challenge from the Challenge Generator at the selected difficulty, manages the circuit-building session by composing the existing simulator interface, invokes Circuit Verifier on submission, and coordinates solution display via Solution Builder.

Difficulty Selector - Screen presented before a challenge begins. Displays three options (Easy, Medium, Hard) with a brief description of each, and initiates a new Challenge Session upon selection.

Challenge Panel - Overlay opened via the View Challenge button. Displays the challenge truth table with labeled input variable columns and the expected output column for every input combination.

Challenge Controls - Toolbar extension added to the simulator interface during a challenge session. Contains three buttons: View Challenge (opens Challenge Panel), Submit (invokes Circuit Verifier and displays Submission Result), and View Solution (invokes Solution Builder and opens Solution Display).

Submission Result - Feedback panel shown after submission. Displays whether the circuit is correct, the number of gates used, the reference minimum established at generation time, and - if incorrect - the

specific input combinations that produce the wrong output.

Solution Display - Renders the program-generated circuit produced by Solution Builder on a read-only Circuit Canvas instance, alongside its gate count.

Technology - React with TypeScript.

Boolean Formula Games Engine:

Purpose: Implements logic-based games centered around Boolean formulas, including formula evaluation and formula equivalence challenges.
This module is separate from the board game and circuit simulator.

Components:

Expression Evaluator - Evaluates parsed expressions using provided variable assignments.
Uses the Logic Gate Module for logical computation.

Formula Generator - Generates Boolean formulas and variable assignments for use in both formula games. Produces formulas of varying syntactic complexity and difficulty and controlling operand count. For the equivalence game, generates pairs of formulas while controlling whether the pair is logically equivalent or not, ensuring a balanced distribution of both outcomes across game sessions.

Formula Evaluation Mode - Validates player answers for Boolean formulas and calculates score based on correctness and response time.

Formula Equivalence Mode - Determines whether two Boolean formulas are logically equivalent.

Timer Manager - Measure response time.

Scoring Manager - Scored based on correctness and response time.

Evaluation Solution Generator - Given a formula tree and a variable assignment, produces the step-by-step evaluation showing how each sub-expression reduces to a boolean value, from the innermost expressions outward to the final result.

Module technology – TypeScript, Vitest.

Boolean Formula Games UI:

Purpose: A client-side module responsible for rendering the shared interface of both equation game modes. It adapts its layout based on the active game mode: in Evaluation mode it displays one formula with variable inputs, an answer selector, and a submit button; in Equivalence mode it displays two formulas

and a pair of answer buttons("equal" button and "not equal" button).

Components:

Formula Game Session - The top-level component rendered when a formula game session begins. Orchestrates the round loop: generates formulas, manages the timer, receives player answers, and updates scores through the Boolean Formula Games Engine. In online mode, receives formulas from the server and submits answers for server-side scoring.

Formula Display - Displays one Boolean formula in evaluation mode and two formulas in equivalence mode.

Variable Assignment Display - rendered in Evaluation mode only. Displays the formula's variable assignment.

Answer Selector - rendered in Evaluation mode only. A small clickable cell after the equal sign that opens a Bit Selector for the player to select the formula's evaluated result (0 or 1).

Submit Button - rendered in Evaluation mode only. A button that stops the timer and submits the selected answer when clicked.

Equivalence Answer Buttons - Two buttons marked "equal" and "not equal" and the moment you press

one of them the timer stops and the answer selected is the one marked with the button you pressed.

Timer Display - shows the time remaining for the current round.

Score Display - Displays the player's score and the opponent's score in online mode.

Round Result - Feedback shown after each round submission, indicating whether the answer was correct and how many points were earned.

Session Summary - Shown at the end of a session. Displays the final score in solo mode, or both players' scores and the winner in online mode.

Notation Selector - A button that opens a dropdown panel listing the three formula notation styles (Mathematical, Text, C Language). The currently selected style is marked with a checkmark. Clicking any option immediately applies that notation to all formula displays.

Notation Legend - A button that opens a reference table showing all logic gate operators and their corresponding symbols in each of the three notation styles.

Draft Pane - A scratch pad area below the answer controls where the player can write working notes while solving a round.

Draft Keyboard - An on-screen keyboard toggled from the Draft Pane, containing the keys relevant to Boolean formula work.

Evaluation Solution Display - Renders the output of the Evaluation Solver as a numbered sequence of sub-expression evaluations, each showing the sub-formula and its value.

Equivalence Solution Display - Renders the output of the Equivalence Solver: for equivalent pairs, two parallel chains of algebraic steps converging to the same canonical form; for non-equivalent pairs, the counter-example assignment and step-by-step evaluation of both formulas.

Round Review - The full review screen for a past round, containing the formula display, the variable assignment (evaluation mode) or second formula (equivalence mode), an interactive answer area, a "Show Solution" toggle, and an empty draft pane. Has no timer and no scoring.

Truth Table Display - renders a truth table for a Formula, with optional label heading ("Formula A" / "Formula B"). True results are shown in green, false in red.

Module technology – React with TypeScript.

Description of architecture layers:

Client:

Purpose: The client provides the user interface, manages user interactions, renders visual elements, and executes all logic required for single-player and local gameplay.

Contained Modules:

- Board Game Engine (from shared workspace)
- Computer Player Engine
- Logic Circuit Simulator Engine
- Boolean Formula Game Engine (from shared workspace)
- Logic Gate handler (from shared workspace)
- Board Game UI
- Boolean Formula Game UI
- Logic Circuit Simulator UI
- Simulator Challenge Engine
- Simulator Challenge UI

Key Responsibilities:

- Rendering the board, circuits, and formulas
- Handling user input
- Running local game logic
- Displaying animations and feedback

Components:

App - Manages the entire client-side navigation flow of the application. Maintains the current screen state and renders the appropriate screen based on it.

Technologies:

- React with TypeScript
- State management: React Context or Zustand/Redux
- Canvas and SVG for visualization
- HTML/CSS for layout and styling

Server:

Purpose: The server supports online multiplayer functionality and acts as the authoritative source of truth for competitive games.

Contained Modules:

- Board Game Engine (from shared workspace for validation)
- Boolean Formula Game Engine (from shared workspace for validation)
- Logic Gate handler (from shared workspace)

Key Responsibilities:

- Managing game rooms and player connections
- Validating moves and results
- Synchronizing state between clients

Components:

WebSocket Manager - Handles real-time bidirectional communication with connected clients. Assigns a unique ephemeral session identifier (UUID) to each player upon connection and stores the mapping between session identifiers and active WebSocket connections, enabling the server to address individual players throughout a game session.

Matchmaking Queue - Maintains waiting queues for players seeking an online match, organized by game type. For formula games, queues are further divided by difficulty level (Easy / Medium / Hard), ensuring matched players always share the same difficulty setting. When a second player joins the same queue, the Matchmaking Queue pairs the two players, requests a new game room from the appropriate room manager, and notifies both players that their game is ready to begin.

Board Game Room Manager - Creates and manages online board game sessions. Maintains the authoritative board state for each active room, validates each incoming move via the Board Game

Engine, updates player scores, and broadcasts the updated state to both players after each move.

Formula Games Room Manager - Creates and manages online formula game sessions for both the Evaluation and Equivalence game modes. Generates formula rounds at the selected difficulty using the Boolean Formula Games Engine, receives answers from both players, validates and scores them server-side, and advances to the next round once both players have responded.

Technologies:

- Node.js runtime
- Express.js for REST endpoints (game creation, joining)
- WebSocket for real-time updates

Shared:

Purpose: The shared layer contains all modules that are required by both the client and the server. It is implemented as an independent npm workspace within the monorepo and imported by both layers. This eliminates code duplication and ensures that shared logic is always consistent across both sides.

Contained Modules:

- Logic Gate Handler
- Board Game Engine
- Boolean Formula Game Engine

Technologies:

- TypeScript